

stock-alert

加密貨幣與股票的自動掃描・停損通知系統

這份文件說明整套系統的組成、每支程式負責什麼、資料怎麼流動，以及各項門檻參數為什麼設成現在的值。

項目	內容
儲存庫	GGININDERR/stock-alert
執行環境	GitHub Actions（無伺服器，全部靠排程觸發）
資料來源	OKX 公開 API（幣）、Yahoo Finance（股）、Google Sheets（持股）
通知管道	Telegram Bot
程式規模	9 支 Python、約 3,400 行
文件產生日	2026-08-20

免責聲明

本系統純粹依技術面（價格與成交量）做機械式篩選，**不構成投資建議**。所有統計數字都是紙上交易結果，未計入滑價，手續費僅以固定 0.1% 估算，實際交易結果會比報表更差。

1 系統全貌

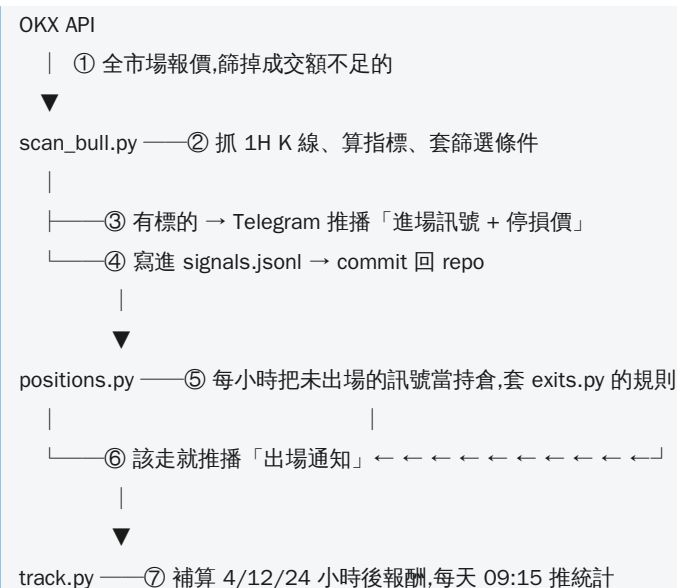
整套系統沒有伺服器。所有動作都由 GitHub Actions 的排程觸發，跑完就結束，狀態只留在 repo 裡的 `signals.jsonl` 這個檔案。這個設計的好處是零維運成本，代價是受制於 GitHub 排程的延遲。

1.1 兩條互不相干的線

系統實際上是兩套東西共用一個 Telegram 機器人：

	幣圈線	股票線
標的	OKX USDT 永續合約 (約 200-260 檔)	你自己的持股清單
做什麼	主動掃描全市場，找出剛啟動的標的	被動監控已持有的部位
頻率	每小時	台股 13:35、美股 21:05 (收盤後)
主程式	<code>scan_bull.py</code> + <code>positions.py</code>	<code>check_stocks.py</code>
資料存放	<code>signals.jsonl</code> (存在 repo 裡)	Google Sheets

1.2 幣圈線的資料流



關鍵在第 ④ 步：訊號寫進檔案並 commit 回 repo，這就是整套系統唯一的「記憶」。因為 GitHub Actions 每次都是全新的容器，不寫回 repo 的東西下一輪就消失了。

1.3 一個重要的先後順序

在 `scan_bull.py` 裡，寫檔案發生在推播之前。這代表 `signals.jsonl` 裡有紀錄、但你手機沒收到，是有可能的——推播失敗不會回頭把紀錄刪掉。查問題時這是判斷「掃描沒掃到」還是「掃到了但沒送出去」的關鍵依據。

2 程式清單

檔案	行數	職責
<code>scan_bull.py</code>	678	幣圈掃描主程式。抓 OKX 報價與 K 線、計算指標、套四種篩選模式、推播進場訊號。Telegram 發送函式也定義在這裡，其他模組都 import 它。
<code>exits.py</code>	235	出場規則的 唯一一份 定義：停損、第一目標、移動停利、訊號失效、時間出場。實盤與回測共用同一份，避免回測驗證的不是實際跑的邏輯。
<code>positions.py</code>	314	持倉監控。把 <code>signals.jsonl</code> 裡未出場的訊號當持倉，每小時抓新 K 線套 <code>exits.py</code> ，該走就推播並把出場結果寫回檔案。
<code>track.py</code>	236	紙上交易追蹤。回頭補算每個訊號 4/12/24 小時後的報酬，產生「這套條件到底準不準」的統計報表。
<code>backtest.py</code>	547	歷史回測。把時間倒回去逐根重算指標，沿用 <code>scan_bull</code> 的篩選條件與 <code>exits.py</code> 的出場規則。支援參數掃描 (<code>--sweep</code>)。
<code>test_backtest.py</code>	250	一致性測試。同一組 K 線餵給 live 版與回測版，逐項比對指標。這是整個專案唯一的自動化測試。
<code>check_stocks.py</code>	580	股票機器人。從 Google Sheets 讀持股，用 Yahoo Finance 查價，算停損停利並推播。也處理 <code>/add /del /list</code> 等持股管理指令。
<code>tg_poll.py</code>	241	Telegram 指令收信。每 5 分鐘去 <code>getUpdates</code> 拉訊息、解析指令、直接執行對應腳本。取代了原本的 Cloudflare Worker。
<code>cf_info.py</code>	91	工具程式。用 Cloudflare API Token 查出 Account ID 與 Worker 清單。只在設定部署時用得到，日常不執行。

`cloudflare_worker.js` 已經不用了

repo 裡還留著 272 行的 `cloudflare_worker.js`，那是舊架構：Telegram → Cloudflare Worker → 觸發 workflow。現在改由 `tg_poll.py` 定時收信，Worker 那條路已經沒有在跑。留著只是備查，真要即時回應（不想等 5-20 分鐘）才需要回頭用它。

3 幣圈掃描：四種模式

掃描分三層：先用 24h 成交額篩掉太薄的標的，再抓 1H K

線算指標，最後套用模式對應的條件組。四種模式抓的是同一件事的不同階段。

模式	抓什麼	成交額門檻	量比門檻	最小停損
early 壓縮後突破	還沒噴的位置。先橫盤、布林帶寬收窄，價格剛站上箱頂（差 0.5% 內也算）。不看漲幅，只看盤整結構有沒有被打破。	1M	2.0	0.3%
breakout 剛啟動	1h / 4h / 24h 三個時間尺度都在動，且貼著前高（48 根內），但漲幅還在合理區間（24h 在 +5~40% 之間）沒噴完。	2M	2.0	1.0%
chase 追高順勢	已經在噴的（24h +20~80%）。門檻最嚴，量比要 3 以上。不是叫你追，是供觀察回踩。	5M	3.0	1.0%
classic 多頭排列 + 量比	最寬鬆的一組：均線多頭排列 + 量比放大 + 有動能。原始版本。	0.3M	2.0	1.0%

3.1 核心指標：量比

量比是整套系統最關鍵的單一條件，也是實務上最常卡住的那道門檻。定義是：

base = 前 20 根 K 棒的平均成交量（不含當根，避免自己稀釋自己）

volr = 最近一根已收盤 K 棒的成交量 ÷ base

注意「已收盤」：程式抓回 K 線後會把最新那根（還在跑、量只累積了一半）直接丟掉，用前一根完整的來算。沒有這道處理的話，每次掃描都會低估量比。

3.2 判讀標籤怎麼來的

推播訊息上的顏色標籤，是量比與「離 MA20 乖離」兩個數字交叉出來的：

標籤	條件	含意
● 末端加速	乖離 $\geq 15\%$	追高性價比差，建議等回踩 MA20 附近
● 剛啟動	量比 ≥ 4 且乖離 $< 10\%$	量能猛、位階乾淨，最理想的進場位置
● 放量但偏高	量比 ≥ 4 且乖離 $\geq 10\%$	資金介入明顯，但已離短均一段，分批比一次進好
● 溫和啟動	量比 < 4 且乖離 $< 10\%$	量能剛放大、位階乾淨，留意後續是否續攻
● 觀察	其餘	多頭整理中還沒噴，先放觀察名單

實測結果與標籤名稱不一致

截至 8/20 的追蹤統計，「● 剛啟動」37 筆平均 -1.06%、勝率 45.9%，反而是「● 溫和啟動」74 筆平均 +1.83%、勝率 56.8% 比較好。量比分組也一樣：量比 2-3 的表現（+3.78%）明顯優於量比 ≥ 10 （-0.68%）。換句話說量比越大不代表越好，過猛反而是末端的訊號。標籤文字目前還沒依這個結果調整。

4 出場規則 (exits.py)

掃描器只回答「進不進場」，出場全部由 exits.py 決定。一根 K 棒內的判定依序是：

順序	規則	判定方式
1	停損	棒內最低價碰到停損價就出場，不等收盤
2	第一目標	獲利達 1R 時出掉一半，剩下的停損拉到成本價
3	移動停利	出過第一批後，停損改跟著「持倉最高價 - 2×ATR」往上抬
4	訊號失效	進場理由消失（跌回箱頂 / MA20 轉弱 / RSI 由熱轉弱）。收盤才判定，避免被棒內雜訊掃出去
5	時間出場	抱滿 24 根 K 棒還沒被觸發就平倉——資金卡住也是成本

4.1 停損價怎麼算

以 ATR 為主、結構（訊號那根 K 棒的低點）為輔，取兩者中離進場價較遠的那個。

理由：純結構停損常常貼價格太近，一根雜訊就被掃掉；用 ATR（1.5 倍）當下限，可以確保停損至少有一個波動幅度的呼吸空間。

4.2 為什麼沒有「停利目標價」

這套設計刻意不設固定停利點，而是把停損往上推：到 1R 出一半並移到成本價（這筆單從此不會賠），之後改用移動停利跟著跑。所以除了「第一目標」以外，所有獲利了結最後都是以停損的形式成交，只是報表上名字不同（保本出場 / 移動停利）。

4.3 最小停損距離：為什麼 early 和其他模式不一樣

停損距離太窄的訊號會直接被丟掉。理由是算術而非回測：手續費來回約 0.2%，停損距離若只有零點幾%，一個跳動就掃掉，期望值必為負。

但這個值不能全模式共用。early

專挑「箱體收窄、帶寬壓縮」的標的，低波動是它的進場條件而非缺陷，停損距離天生就窄（實盤中位數 1.0%，breakout 是 4.5%）。用通用的 1.0% 會砍掉 early 近半訊號，所以它降到 0.3%，只擋掉「波動死透」的極端值——例如代幣化美股在美股收盤後，量比會因為均量極小而虛高，ATR 卻小到停損只剩零點幾 %。

5 排程

全部由 GitHub Actions 的 cron 驅動。cron 一律是 UTC，台北時間要加 8 小時。

Workflow	cron (UTC)	台北時間	做什麼
Crypto Bull Scan scan_bull.yml	0 * * * *	每小時	breakout + early 掃描、出場監控、補算報酬
	30 */4 * * *	每 4 小時	備援班（見下方說明）
	15 1 * * *	09:15	每日訊號追蹤統計推播
Telegram Bot tg_bot.yml	*/5 * * * *	每 5 分鐘	收 Telegram 指令並執行
Stock Bot stock_check.yml	35 5 * * 1-5	13:35 平日	台股收盤後檢查持股
	5 21 * * 1-5	05:05 平日	美股收盤後檢查持股
Backtest backtest.yml	(手動)	—	一致性測試 + 歷史回測
Cloudflare Info cf_info.yml	(手動)	—	查 Cloudflare 帳號資訊

5.1 為什麼有兩條 cron 做同一件事

這是踩過坑之後的補救。原本用 `github.event.schedule` 判斷是哪一條 cron 觸發的，結果 early 幾乎不跑：GitHub 的排程會延遲，04:00 那班常常拖到 04:32 才送達，而它帶的參數仍然是原本那條 `0 * * * *`，於是 early 被跳過；真正的 `30 */4` 那班則整個被丟掉。實測 2026-08-19 早上三次排程全部漏掉 early。

現在改成不看是哪條 cron 觸發，改看實際時鐘（`date -u +%H`）。兩條 cron 都留著當備援，誰先送到都算數。

GitHub 排程不保證準時，也不保證會跑

這是整套系統最大的不確定性來源。實際觀察到的觸發時間散佈在整點後 0-58 分之間，偶爾整個小時被跳過（例如 8/20 的 01:00 UTC 那班就沒跑）。任何依賴「準點觸發」的邏輯都會不定期失效。

5.2 每日心跳

沒掃到標的時系統預設靜默，否則一天會收到 24 則「本輪無標的」。例外是 UTC 00 時（台北 08:00）那班會帶 `--always` 參數，不管有沒有標的都推一則，當作「系統還活著」的心跳。

這裡有個已知的脆弱點（尚未修）

心跳的判斷是 `h=$(date -u +%H)`，只有整點小時剛好等於 00 才發。8/20 那班是 00:58 UTC 才被觸發，差 2 分鐘就會落到 `h=01`，心跳整個靜默跳過。真的發生時，你早上會連「本輪無標的」都收不到，看起來就像系統死了。建議改成「當天還沒發過就補發」，不看時鐘。

6 Telegram 指令

指令由 tg_poll.py 每 5 分鐘收一次，所以會延遲 5-20 分鐘才有回應。太舊的訊息（超過 30 分鐘）不執行，避免第一次啟用時把積壓的指令全部重跑。

6.1 幣圈

指令	作用
/scan	立刻掃一次（預設 breakout）
/scan early	壓縮後突破，還沒噴的位置
/scan chase	追高順勢，供觀察回踩
/scan classic	最寬鬆的一組
/pos	目前幣圈持倉：停損價、離停損多遠、是否逼近停損
/stats	訊號追蹤統計（這些條件到底準不準）

6.2 股票

指令	作用
/check /check_all	台股 + 美股，檢查停損與停利
/check_tw /check_us	只檢查單一市場
/list	持股清單 + 總損益
/price <TW US> <代碼>	查現價 + 均線
/add <TW US> <代碼> <成本> <股數> [名稱]	新增持股
/del <TW US> <代碼>	刪除持股
/sold <TW US> <代碼> <股數>	賣出 N 股（自動扣減）
/clear ALL CONFIRM	清空全部持股（需確認字串）

說明文字已經過期

tg_poll.py 裡 /help 回覆的內容還寫著「每 4 小時掃 early」，但 early 早就改成每小時跑了。改動的理由是：early 要抓的是「剛站上箱頂」那一瞬間，不是一段狀態，四小時掃一次等於最多晚四根 K 棒才看到。help 文字忘了跟著更新。

7 實測成效

以下數字取自 2026-08-20 的 track.py 報表，樣本期間 2026-08-13 至 2026-08-20，共 175 筆訊號。

7.1 原始訊號品質（固定時間出場，無停損）

持有時間	筆數	平均	中位	勝率	扣 0.1% 成本
4 小時	174	+0.71%	+0.26%	55.7%	+0.61%
12 小時	170	+1.97%	+0.64%	58.8%	+1.87%
24 小時	123	+1.75%	+0.55%	54.5%	+1.65%

7.2 實際出場（套用 exits.py 的完整規則）

出場原因	筆數	占比	平均報酬
移動停利	64	53.8%	+2.60%
跌回箱頂	22	18.5%	-1.86%
停損	20	16.8%	-2.53%
保本出場	7	5.9%	+0.83%
MA20 轉弱	6	5.0%	-0.56%
合計 119 筆		勝率 59.7%	+0.65%

7.3 依模式

模式	筆數	平均	中位	勝率
early	57	+2.88%	+1.37%	68.4%
breakout	66	+0.77%	-0.90%	42.4%

early 明顯優於 breakout——勝率高 26 個百分點，而且 breakout 的中位數是負的（表示它靠少數大贏家把平均拉起來）。這符合設計直覺：early 抓的是還沒噴的位置，breakout 抓的是已經在動的，後者本質上就是在追價。

7.4 依量比分組

量比區間	筆數	平均	中位	勝率
2 - 3	46	+3.78%	+1.31%	65.2%
3 - 5	52	+1.45%	+0.24%	51.9%
5 - 10	14	-1.95%	-0.61%	42.9%
≥ 10	11	-0.68%	-4.64%	36.4%

這張表是整份報表最反直覺的地方

量比越大，表現越差，而且是單調的。量比 2-3 這組（剛放量）平均 +3.78%、勝率 65.2%；量比 ≥ 10 那組（爆量）平均 -0.68%、中位數 -4.64%、勝率只有 36.4%。但目前 scan_bull.py 的 pick() 是按量比由大到小排序推播的，等於把表現最差的排在最前面。樣本數還小（ ≥ 10 那組只有 11 筆），但方向值得注意。

8 維護與排錯

8.1 「怎麼都沒收到推薦」的判斷流程

沒收到推播有三種可能，用這個順序切開：

- ① 掃描有沒有跑？
 - GitHub Actions 頁面看 Crypto Bull Scan 的執行紀錄
 - 全綠但沒訊號 = 有跑,往下一步
- ② 掃描有沒有掃到東西？
 - 看 signals.jsonl 最後一筆的時間戳
 - 有新紀錄 = 掃到了,問題在推播(往 ③)
 - 沒新紀錄 = 真的沒標的(往 ④)
- ③ 推播是不是失敗了？
 - 看 workflow log 有沒有「已推播 N 檔」這行
 - 修正後的版本失敗會讓 workflow 變紅並印出原因
- ④ 沒標的算不算異常？
 - 看空窗長度,不要看「平均每小時幾筆」(訊號是叢發的,平均看不出來)
 - 歷史空窗中位數 1.0h、平均 2.2h,最長曾經 13h
 - 6 小時以內的空窗一週會出現好幾次,屬正常

8.2 各條件通過數：調門檻的依據

每次掃描的 log 都會印出各條件單獨的通過數，這是判斷「卡在哪一關」最直接的資訊：

```
掃描 199 檔,符合 0 檔
各條件通過數: 多頭排列 77 | MA20向上 153 | 量比>2 7 | 1h +1~10% 17
               | 4h +3~20% 19 | 24h +5~40% 105 | 距前高<2% 54
               | RSI 50-78 148
```

上面這組 (8/20 台北 12:32) 可以直接讀出：趨勢面條件都過一大半 (MA20 向上 153 檔、RSI 148 檔)，單一瓶頸是量比只有 7 檔過關。要放寬就從量比下手，其他條件動了也沒用。

8.3 改動篩選邏輯的規矩

回測 (backtest.py) 直接 import scan_bull 的 conds_* 與 exits.py，不另外抄一份。這是刻意的：兩邊若各寫一份，回測驗證的就不是實際會通知你的那套邏輯。

但 live 版只算最後一根 K 棒的指標，回測版算整條序列，實作方式不同。兩者的一致性由 test_backtest.py 保證——它把同一組 K 線餵給兩邊，逐項比對。改任何指標計算都要跑它：

```
python test_backtest.py      # 全部一致 ✓ 才算過
```

8.4 目前已知的待辦

項目	狀態
推播失敗被靜默吞掉 send_telegram 回傳 resp.ok，但十幾個呼叫端沒有一個檢查回傳值，也沒有任何 log。掃到標的 → 寫進 signals.jsonl → 推播被打回 → 仍然 exit 0 → workflow 全綠。訊號記錄裡有，手機上沒有，且查不出來。	已修， 待合併
心跳的時鐘判斷太脆弱 h=\$(date -u +%H) 遇到 GitHub 排程延遲會整個漏掉當天的心跳。8/20 那次差 2 分鐘中招。	未修
/help 文字過期 還寫著「每 4 小時掃 early」，實際已改每小時。	未修
推播排序與實測結果相反 pick() 按量比由大到小排序，但量比越大實測表現越差。	待討論
判讀標籤與實測不符 「● 剛啟動」實測平均 -1.06%，反而比「● 溫和啟動」(+1.83%) 差。	待討論

— 文件結束 —